Post-processing

Two Python scripts are bundled in `test/` of the SWAT+ source repository. Each is a small standalone tool. They are not installed by the build; copy them into your working tree or invoke them directly.

test/spcheck.py

A regression-test driver. Runs a SWAT+ executable against a scenario directory, then performs a field-by-field numerical comparison against a golden output.

Subcommands

`spcheck.py` has three subcommands.

run

`spcheck.py run <executable> <scenario>`

Workflow:

1. Looks up `data/<scenario>`.
2. Creates `runs/<scenario>/<NN>#<timestamp>#<executable>/` and copies the scenario into it.
3. Runs `<executable>` (from `build/<executable>`) with that directory as cwd.
4. If the executable returns non-zero, renames the run directory to end in `-FAILED`.

`NN` is the next two-digit serial number (`01`, `02`, …).

compare

`spcheck.py compare <scenario> <a> <b> [--abserr 1e-8] [--relerr 0.05] [--nlines N] [--nerrorlines N]`

Compares the outputs of two prior runs of the same scenario, identified by their serial numbers. Reads the list of output files from `<scenario>/.testfiles.txt` (one filename per line, `#` starts a comment).

For each pair of files, line-by-line, field-by-field:

- Splits the line into tokens by whitespace.
- Skips non-numeric tokens. Skips lines containing `MODULAR` (the version banner).
- Compares each float field: a difference is flagged when

  `abs(f1 - f2) >= aerr + rerr * abs(f2)`
- Defaults: `aerr = 1e-8`, `rerr = 0.05` (5 percent).
- For each diff, prints the line numbers, the field positions, the two values, and the absolute and relative error.

ctest

`spcheck.py ctest <model_path> <data_dir> <tmp_dir> [--abserr ...] [--relerr ...] [--nlines N] [--nerrorlin`

For automated test runs. Copies `<data_dir>` into `<tmp_dir>/<scenario_name>`, runs `<model_path>` in that tmp directory, then compares the newly produced outputs against the originals in `<data_dir>`. Exits non-zero if any comparison errors are above the thresholds.

What it is good for

- Detecting numerical regressions when changing source code.
- Comparing two executables (different compilers, different revisions) against the same scenario.
- Running CI tests with a small abserr/rerr tolerance to flag any unexpected drift.

What it does not do

- Does not understand the column structure of SWAT+ outputs. Tokens are compared by position only.
- Skips integer fields (calls `int()` first; floats only).
- Trusts that both files have the same line count and the same number of float tokens per line. Differences in structure produce a `danger` return code (-1) for the line and the comparison stops on that line.

test/swat_output_to_csv_files.py

A pandas-based parser for SWAT+ text outputs. Maps every known output filename to a read specification (rows to skip, header row, column names, separator), then loads it with `pandas.read_csv`.

How it is structured

A single module-level dictionary `spec_dict` keys every supported filename to a dict of `pandas.read_csv` arguments.

Two default specifications are reused for most files:

```
default_spec = {
    "skiprows": [0, 2],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}

default_spec2 = {
    "skiprows": [0],
    "header":   [0],
    "column_names": None,
    "delim_whitespace": True
}
```

`default_spec` skips line 0 (the SWAT+ banner) and line 2 (the units row), so line 1 becomes the column header. This matches the standard SWAT+ output layout: title, header, units, data.

`default_spec2` keeps the units row in the data. Used for files that do not have a units row (`lu_change_out.txt`, `basin_crop_yld_*.txt`).

`crop_yld_aa.txt` and `crop_yld_yr.txt` get `skiprows=2` (no list) because their banner is two lines long.

`checker.out` and `files_out.out` get explicit `column_names` lists because the file headers do not parse cleanly as a single row of column names.

Files covered

The script supports these outputs out of the box:

```
files_out.out, checker.out
hru_orgc.txt, hru_wb_aa.txt, hru_ncycle_aa.txt, hru_nb_aa.txt,
hru_carbon_aa.txt, hru_nut_carb_gl_aa.txt, hru_ls_aa.txt, hru_pw_aa.txt
basin_wb_aa.txt, basin_nb_aa.txt, basin_carbon_aa.txt,
basin_ls_aa.txt, basin_pw_aa.txt
crop_yld_yr.txt, crop_yld_aa.txt
lu_change_out.txt, basin_crop_yld_yr.txt, basin_crop_yld_aa.txt
hydout_aa.txt, hydin_aa.txt
```

```
deposition_aa.txt, wetland_aa.txt
basin_aqu_aa.txt, basin_res_aa.txt
recall_aa.txt
basin_cha_aa.txt, basin_sd_cha_aa.txt, basin_sd_chamorph_aa.txt
basin_psc_aa.txt
ru_aa.txt
```

Entry point

```python
def run():
    fname = "files_out.out"
    df = read_output(fname, spec_dict, True)
    for index, row in df.iterrows():
        fname = row["filename"]
        if fname == "mgt_out.txt" or fname == "yield.out":
            continue
        df = read_output(fname, spec_dict, True)
```

When run as `python swat_output_to_csv_files.py`, the script:

1. Reads `files_out.out` from the current directory.
2. For every output filename listed there (other than `mgt_out.txt` and `yield.out`), loads the file with the matching spec and writes a tab-separated `.csv` alongside the original.

`read_output(fname, spec_dict, write_csv=False)` is the unit entry point. Pass `write_csv=True` to write `<fname>.csv`; the function always returns the loaded DataFrame.

Limitations

- The script must be run in the directory containing the SWAT+ outputs.
- If a filename is not in `spec_dict`, the script prints `Swat+ output filename not found in read specification dictionary.` and calls `exit(1)`.
- `pandas.read_csv(..., delim_whitespace=True)` is deprecated in newer pandas versions in favour of `sep=r"\s+"`. Expect a `FutureWarning`.
- The output `.csv` files are tab-separated despite the `.csv` extension (`output_seperator = "\t"`).

Typical usage

```
cd <scenario directory>
python /path/to/swat_output_to_csv_files.py
# produces hru_wb_aa.txt.csv, basin_wb_aa.txt.csv, ... in the same dir
```

Or, in a notebook, import the module and call `read_output` directly on the file of interest to get a DataFrame in memory:

```python
import swat_output_to_csv_files as s
df = s.read_output("basin_wb_aa.txt", s.spec_dict, write_csv=False)
```

Related files

- Checker and diagnostics. `files_out.out` is the entry point for the CSV script.
- Naming convention. Explains which file is which.